

Introduction to programming

Labor07

Revision

- ▶ Subroutines
 - Procedures
 - Functions
 - ▶ Variables
 - Local
 - Global
 - ▶ Parameter passing
 - ▶ Optional parameters
 - ▶ `main()` function
- 
- A decorative gradient bar at the bottom left of the slide, transitioning from dark red to light grey.

Main function

- ▶ Entry point to the program:

```
def main():  
    pass
```

```
if __name__ == "__main__":  
    main()
```

Homework 1

<https://viskillz.inf.unideb.hu/prog/#/?week=P1041>

- ▶ Write a function named *count_of_local_minimums()* that returns the number of elements in a list that are smaller than **both of their neighbours**.
- ▶ In case of the *main()* function
 - Read list elements until a given stopping condition.
 - Call *count_of_local_minimums()* and print the number of local minimums.
- ▶ Input handling options:
 - Read lists until EOF
 - Read **n** test cases (lists)
 - Read until an empty line

Homework 1

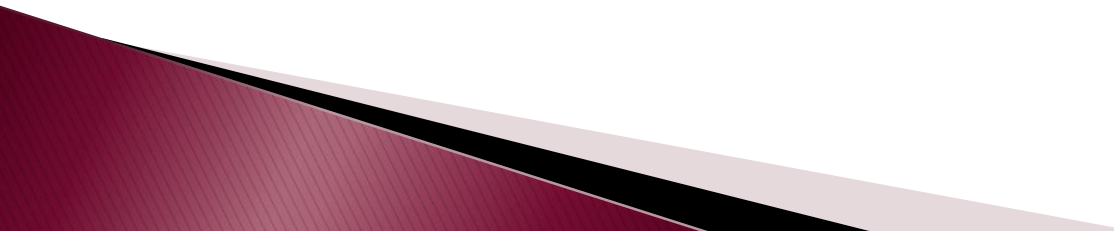
<https://viskillz.inf.unideb.hu/prog/#/?week=P1041>

Sample Input

```
97 -52 -12 41 87 19 35 -45 43
18 92 -21 -86 65
-60 -5 -73 -96 -31 1 72
-42 2 -67 -1 -35 46 32 -46 71 -56
-65 -63 3 -60 -14 61 41 -53
-60 84 47 -76 -68 4
```

Output for Sample Input

```
3
1
1
3
1
1
```



Local minimums

```
def count_of_local_minimums(numbers: list[int]) -> int:
    if len(numbers) < 3:
        return 0

    count = 0
    for i in range(1, len(numbers) - 1):
        if numbers[i - 1] > numbers[i] < numbers[i + 1]:
            count += 1

    return count
```

Local minimums

```
def count_of_local_minimums(numbers: list[int]) -> int:
    if len(numbers) < 3:
        return 0

    count = 0
    for i in range(1, len(numbers) - 1):
        if numbers[i - 1] > numbers[i] and
            numbers[i] < numbers[i + 1]:
            count += 1

    return count
```

Read until EOF main() function

```
for line in sys.stdin:  
    numbers = [int(number) for number in line.split()]  
    print(count_of_local_minimums(numbers))
```


Read n test cases (lists)

main() function

```
n = int(input())
for i in range(n):
    line = input()
    numbers = [int(number) for number in line.split()]
    print(count_of_local_minimums(numbers))
```

Read until an empty line

main() function

```
line = input()
while line != '':
    numbers = [int(number) for number in line.split()]
    print(count_of_local_minimums(numbers))
    line = input()
```

Homework 2

<https://viskillz.inf.unideb.hu/prog/#/?week=P1042>

- ▶ Write a function called *is_palindrome()* that checks whether a list of integers is a **palindrome**.
- ▶ In the *main()* function:
 - Read lists until the given stopping condition.
 - Call *is_palindrome()* and print **True** or **False** depending on whether the list is a palindrome.
- ▶ Input handling options:
 - Read until EOF
 - Read n test cases (lists)
 - Read until an empty line

Homework 2

<https://viskillz.inf.unideb.hu/prog/#/?week=P1042>

Sample Input

1 2 3 4 5 6

6 5 4 3 2 1

1 2 3 2 1

1 1 1 1

1

-1

1 2 3 5 8 13

Output for Sample Input

False

False

True

True

True

True

False

Palindrome

```
def is_palindrom(numbers: list[int]) -> bool:  
    if numbers[::-1]==numbers[::1]:  
        return True  
    else:  
        return False
```

Palindrome

```
def is_palindrom(numbers: list[int]) -> bool:
    for i in range(len(numbers) // 2):
        if numbers[i] != numbers[-1 - i]:
            return False
    return True
```

Palindrome

```
def is_palindrom(numbers: list[int]) -> bool:
    for i in range(len(numbers) // 2):
        if numbers[i] != numbers[len(numbers)-1-i]:
            return False
    return True
```

Read lists until EOF

main() function

```
for line in sys.stdin:  
    numbers = [int(number) for number in line.split()]  
    print(is_palindrome(numbers))
```


Read n test cases (lists)

main() function

```
n = int(input())
for i in range(n):
    line = input()
    numbers = [int(number) for number in line.split()]
    print(is_palindrome(numbers))
```

Read until an empty line

main() function

```
line = input()
while line != '':
    numbers = [int(number) for number in line.split()]
    print(is_palindrome(numbers))
    line = input()
```

Homework 3

<https://viskillz.inf.unideb.hu/prog/#/?week=P1052>

- ▶ Write a function called ***double_even_digits()*** that duplicates every **even digit character** in the given string and returns the new string.
- ▶ In the *main()* function:
 - Read input strings until the given stopping condition.
 - Call ***double_even_digits()*** and print the result.
- ▶ Input handling options:
 - Read strings until EOF
 - Read n test cases
 - Read until an END string

Homework 3

<https://viskillz.inf.unideb.hu/prog/#/?week=P1052>

Sample Input

```
0123456789
a b c d e f g h
aB1cD2eF3gH4
```

Output for Sample Input

```
001223445667889
a b c d e f g h
aB1cD22eF3gH44
```

Double even digits

```
def double_even_digits(original: str) -> str:  
    r = ''  
    for c in original:  
        if c in '02468':  
            r = r + 2 * c  
        else:  
            r = r + c  
    return r
```

Double even digits

```
def double_even_digits(original: str) -> str:  
    return ''.join([c * 2 if c in '02468'  
                    else c for c in original])
```

```
def main1(): # Read strings until EOF
    for line in sys.stdin:
        print(double_even_digits(line.strip()))
```

```
def main2(): # Read n test cases
    n = int(input())
    for i in range(n):
        line = input()
        print(double_even_digits(line))
```

```
def main3(): # Read until an END string
    line = input()
    while line != 'END':
        print(double_even_digits(line))
        line = input()
```

```
if __name__ == '__main__':
    main3()
```

Recursive functions

- ▶ A **recursive function** is a function that **calls itself** within its own body.
- ▶ A recursive algorithm is typically used when a complex problem can be divided into smaller subproblems of the same type.
- ▶ A recursive solution usually consists of:
 - True branch – stop condition
 - False branch – recursive call
- ▶ Recursive algorithms are usually used when the original task is too complicated, but with recursive calls we can reduce this to simpler (sub)tasks
- ▶ Accordingly, a recursive task is usually defined in a similar way:
 - trivial solution
 - simplification of a general case

Recursive functions

► Advantages:

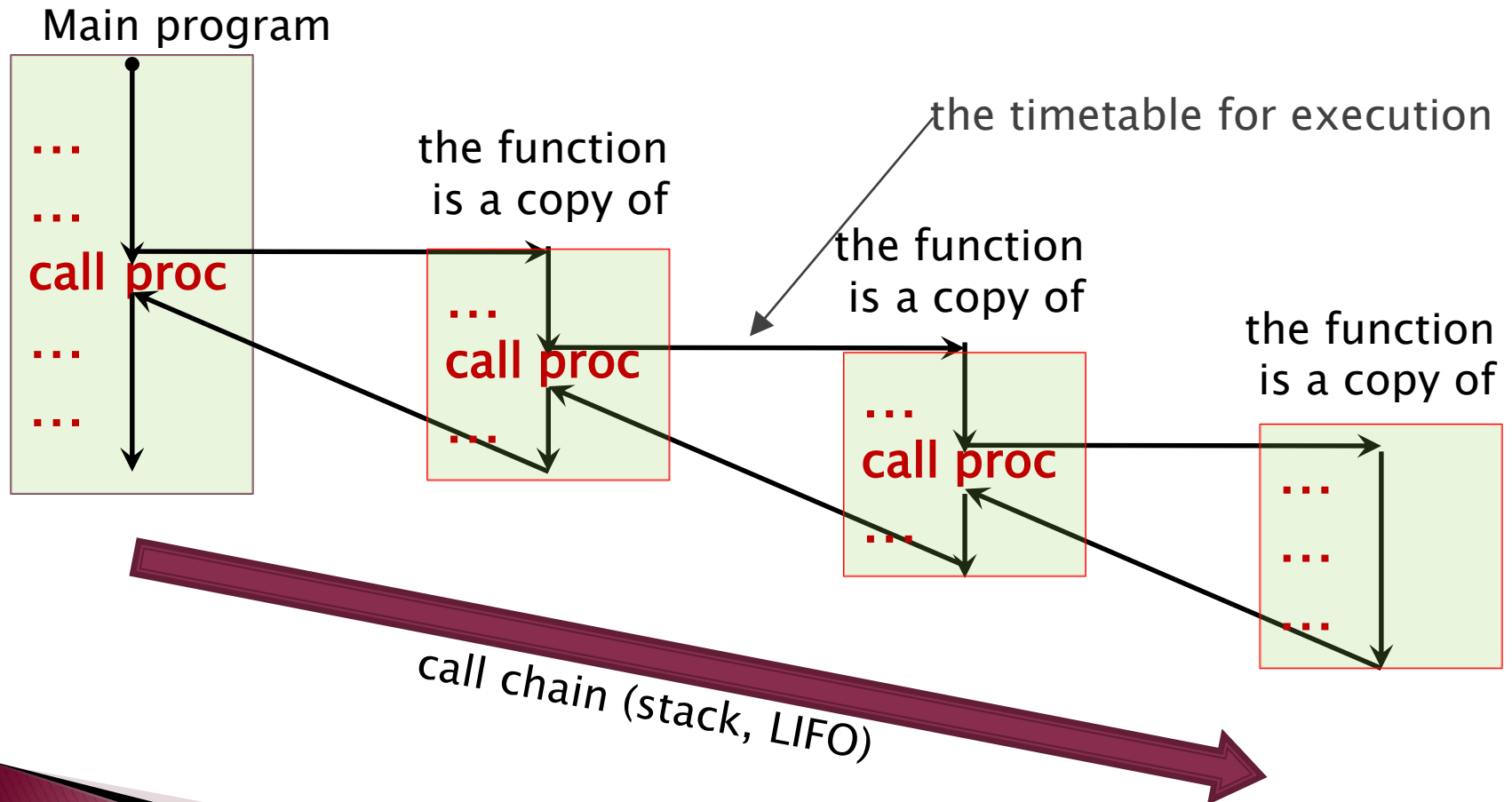
- Makes code more elegant and readable for certain problems.
- Complex problems can be expressed as smaller, simpler ones.
- Some problems (e.g., tree traversal, fractals) are more naturally expressed recursively.

► Disadvantages:

- Harder to follow and debug than iterative solutions.
- Recursion can be **memory- and time-intensive** (each call uses stack space).
- Recursive functions are difficult to debug.

Recursion

- ▶ A subprogram calls itself



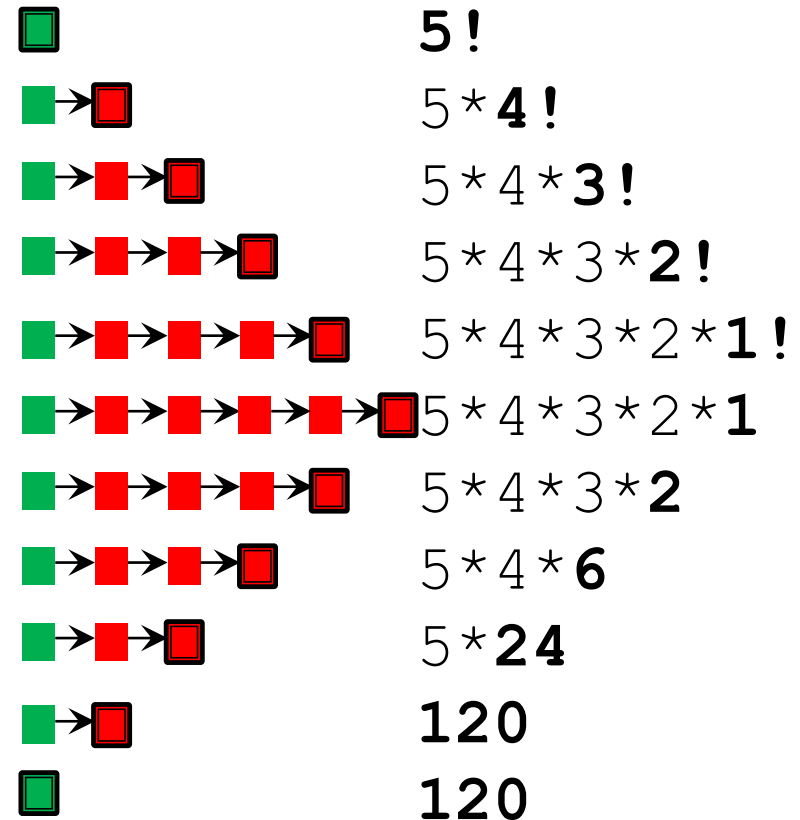
Exercise

- ▶ Write a recursive function that determines the factorial of N .

Recursion

► Factorial:

- If the (N) is 1, than the factorial is 1.
- Else $N! = N * (N-1)!$



Solution1

```
def n_factorial(n: int) -> int:  
    if n == 1:  
        return 1  
    else:  
        return n * n_factorial(n - 1)
```

```
def main():  
    print(n_factorial(5))
```

```
if __name__ == "__main__":  
    main()
```

Solution2

```
def n_factorial(n: int) -> int:  
    return 1 if n == 1 else n_factorial(n - 1) * n
```

```
def main():  
    print(n_factorial(5))
```

```
if __name__ == "__main__":  
    main()
```

```
import sys

def n_factorial(n: int) -> int:
    if n == 1:
        return 1
    else:
        return n * n_factorial(n - 1)

def n_factorial1(n: int) -> int:
    return 1 if n == 1 else n *
        n_factorial1(n - 1)

def read() -> None:
    # read until EOF
    while True:
        try:
            n = int(input())
            print(n_factorial(n))
        except EOFError:
            break
```

```
def read1() -> None:
    # read until EOF
    for line in sys.stdin:
        n = int(line)
        print(n_factorial(n))

def read2() -> None:
    # N test cases
    n=int(input())
    for i in range(n):
        num=int(input())
        print(n_factorial(num))

def read3() -> None:
    # numbers between [0,50]
    while True:
        n=int(input())
        if n<0 or n>50:
            break
        print(n_factorial(n))

def main():
    read3()

if __name__ == '__main__':
    main()
```

Exercise

- ▶ Write a recursive function that computes X^n .

Solution

```
def power(x: int, n: int) -> int:  
    if n == 1:  
        return x  
    else:  
        return x * power(x, n - 1)
```

```
def main():  
    x = int(input("x = "))  
    n = int(input("n = "))  
    print(power(x, n))
```

```
if __name__ == "__main__":  
    main()
```

Solution

```
def power(x: int, n: int) -> int:  
    return x if n == 1 else x * power(x, n - 1)
```

```
def main():  
    x = int(input("x = "))  
    n = int(input("n = "))  
    print(power(x, n))
```

```
if __name__ == "__main__":  
    main()
```

```
import math
def power_it(x: int, n: int) -> int:
    p = 1
    for i in range(n):
        p = p * x
    return p

def power_rec1(x: int, n: int) -> int:
    if n == 1:
        return x
    else:
        return x * power_rec1(x,n-1)

def power_rec2(x: int, n: int) -> int:
    return x if n == 1 else x * power_rec2(x,n-1)

def main():
    x=int(input("x="))
    n=int(input("n="))
    print(power_it(x,n))
    print(power_rec1(x, n))
    print(power_rec2(x, n))
    print(int(math.pow(x,n)))
if __name__ == '__main__':
    main()
```

Recursion

- Write a recursive function that returns the sum of numbers from 1 to N.

$$\sum_{i=1}^n i$$

Solution

```
def n_sum(n: int) -> int:
    if n == 1:
        return 1
    else:
        return n + sum(n - 1)

def main():
    print(n_sum(n))

if __name__ == "__main__":
    main()
```

Solution

```
def n_sum1(n: int) -> int:  
    return 1 if n == 1 else n + sum1(n - 1)  
  
def main():  
    print(n_sum1(n))  
  
if __name__ == "__main__":  
    main()
```

Recursion

- Write a recursive function that determines the sum of the squares from 1 to N.

$$\sum_{i=1}^n i^2$$

Solution

```
def sum2(n: int) -> int:  
    if n == 1:  
        return 1  
    else:  
        return n * n + sum2(n - 1)
```

```
def sum2(n: int) -> int:  
    return 1 if n == 1 else n * n + sum2(n - 1)
```


Exercise

- Write a recursive function that calculates the following series:

$$\sum_{i=1}^n (i^2 - 2)$$

Solution

```
def sum3(n: int) -> int:  
    if n == 1:  
        return -1  
    else:  
        return n * n - 2 + sum3(n - 1)
```

```
def sum3(n: int) -> int:  
    return -1 if n == 1 else n * n - 2 + sum3(n - 1)
```

Exercise

- Write a recursive function that calculates the following series:

$$\sum_{i=3}^n (i^2 - 2)$$

Solution

```
def sum4(n: int) -> int:  
    if n == 3:  
        return 7  
    else:  
        return n * n - 2 + sum4(n - 1)
```

```
def sum4(n: int) -> int:  
    return 7 if n == 3 else n * n - 2 + sum4(n-1)
```

Exercise

- Write a recursive function that calculates the following series:

$$\sum_{i=4}^n i(i^2 - 1)$$

Solution

```
def sum5(n: int) -> int:  
    if n == 4:  
        return 60  
    else:  
        return n * (n * n - 1) + sum5(n - 1)
```

```
def sum5(n: int) -> int:  
    return 60 if n == 4 else n * (n * n - 1) + sum5(n-1)
```

Exercise

- Write a recursive function that calculates the following series:

$$\sum_{i=2}^n (i^3 - 5)$$

Solution – Recursive

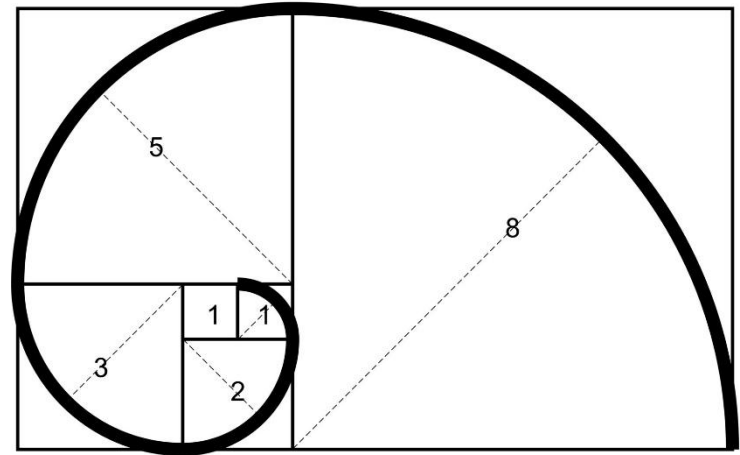
```
def sum6(n: int) -> int:  
    if n == 2:  
        return 3  
    else:  
        return n * n * n - 5 + sum6(n - 1)
```

```
def sum6(n: int) -> int:  
    return 3 if n == 2 else n * n * n - 5 + sum6(n-1)
```


Solution – Iterative

```
def sum6(n: int) -> int:  
    sum = 0  
    for i in range(2, n + 1):  
        sum += i * i * i - 5  
    return sum
```

Exercise



- Write a recursive function that returns the *n-th Fibonacci number*.

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Solution – Recursive

```
def fibonacci(n: int) -> int:  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci (n - 1) + fibonacci (n - 2)
```

Solution – Iterative

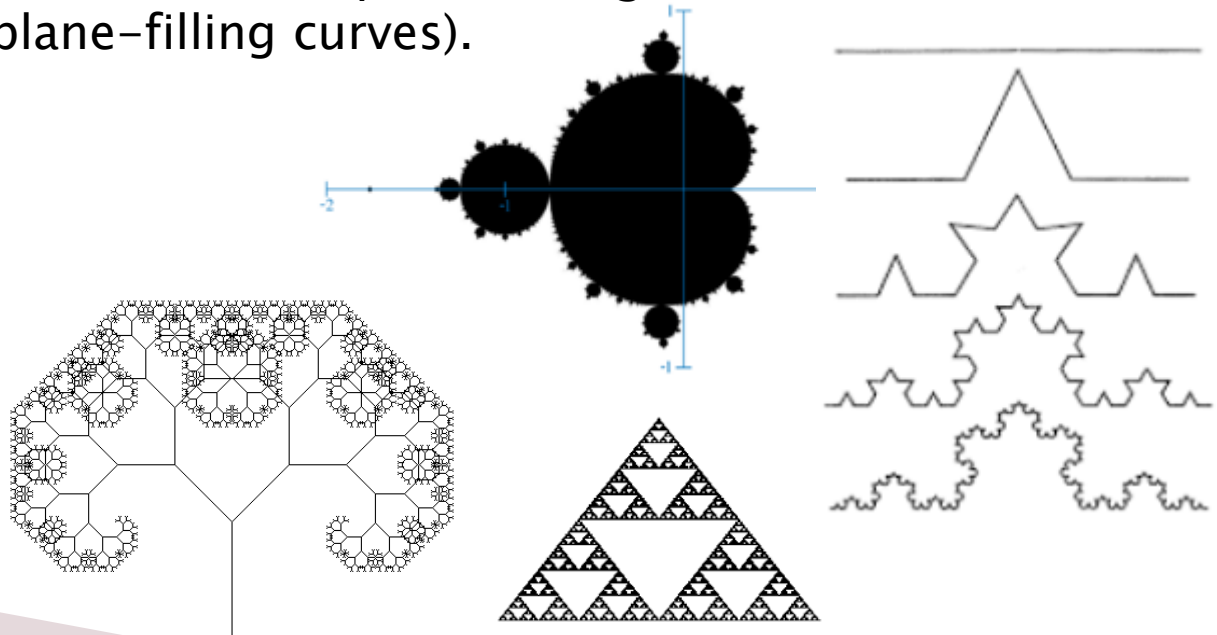
```
def fibonacci(n):  
    f1 = 0  
    f2 = 1  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        for i in range(1, n):  
            f3 = f2 + f1  
            f1 = f2  
            f2 = f3  
        return f3
```

Solution – Iterativ – List

```
def fibonacci1(n: int) -> list[int]:  
    #result/return value -> list  
    f = [0, 1]  
    for i in range(2, n):  
        f.append(f[i - 1] + f[i - 2])  
    return f  
  
print(*fibonacci1(n)) n Fibonacci number
```

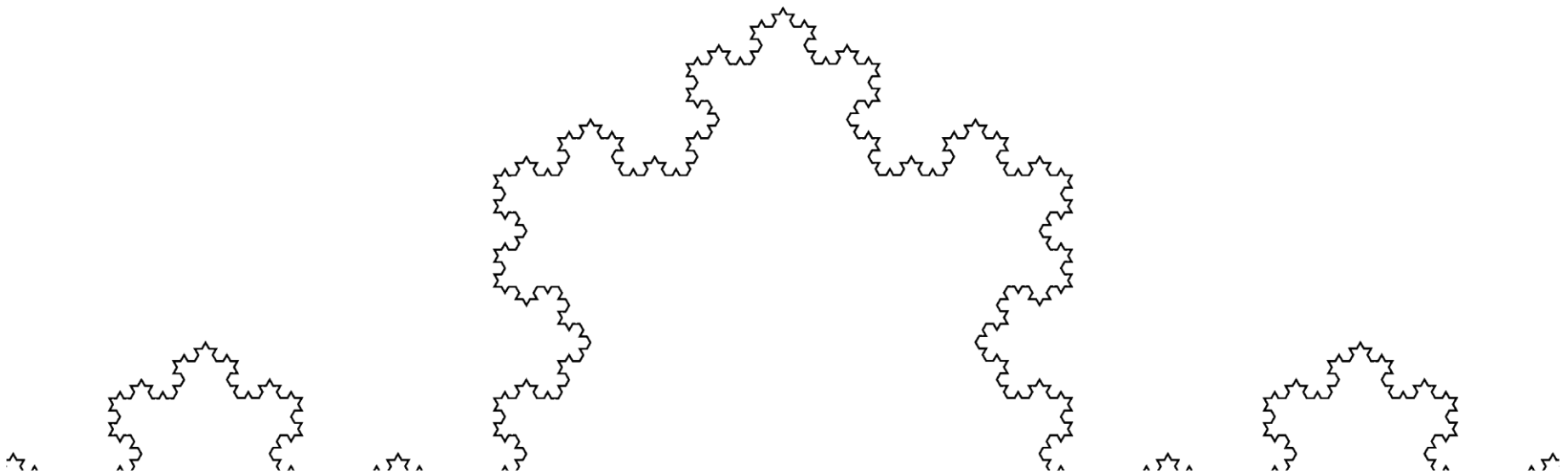
Applications of recursion

- ▶ Fractals
- ▶ Fractals are "self-similar", infinitely complex mathematical shapes, whose varied forms have at least one recurrence that is recognisable (i.e. describable by mathematical means).
- ▶ The name was given in 1975 by Benoît Mandelbrot, from the Latin word fractus (meaning broken; fracture), which refers to the fractional number of dimensions of such shapes, although not all fractals are fractional (such as plane-filling curves).
- ▶ Koch curve
- ▶ Sierpinski triangle
- ▶ Mandelbrot set
- ▶ Pythagorean tree
- ▶ Newton fractal
- ▶ Julia set



Koch curve

- ▶ The **Koch curve** (or snowflake) is a fractal created by repeatedly dividing a line segment into three equal parts and replacing the middle segment with two sides of an equilateral triangle.



Koch curve

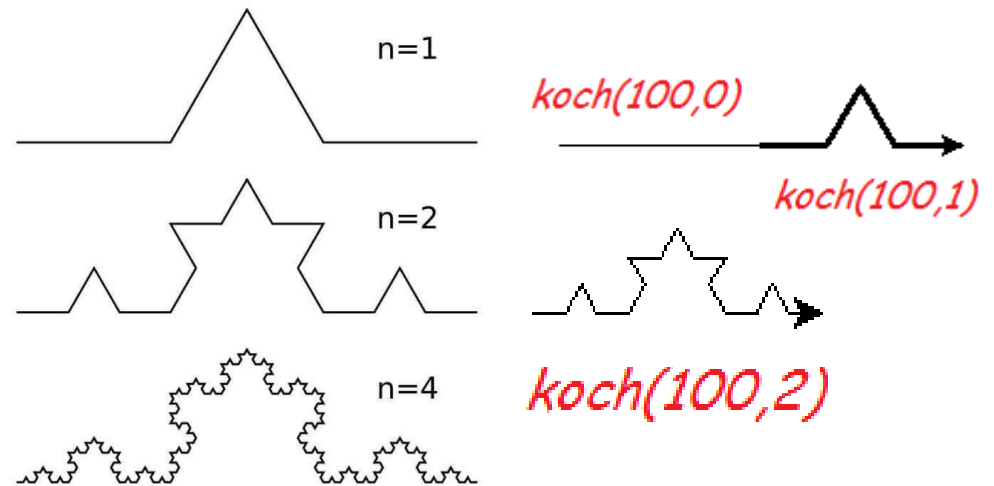
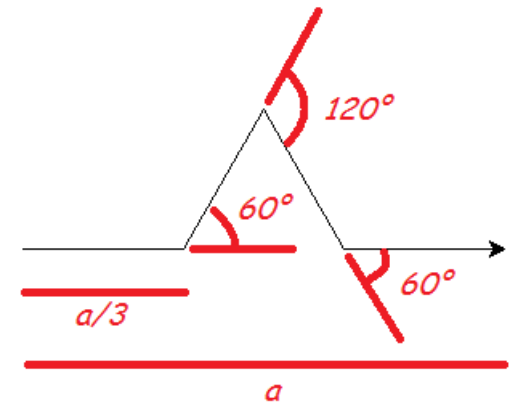
```
from turtle import *
```

```
def koch(a, order):  
    if order > 0:  
        for t in [60, -120, 60, 0]:  
            forward(a/3)  
            left(t)
```

```
    else:  
        forward(a)
```

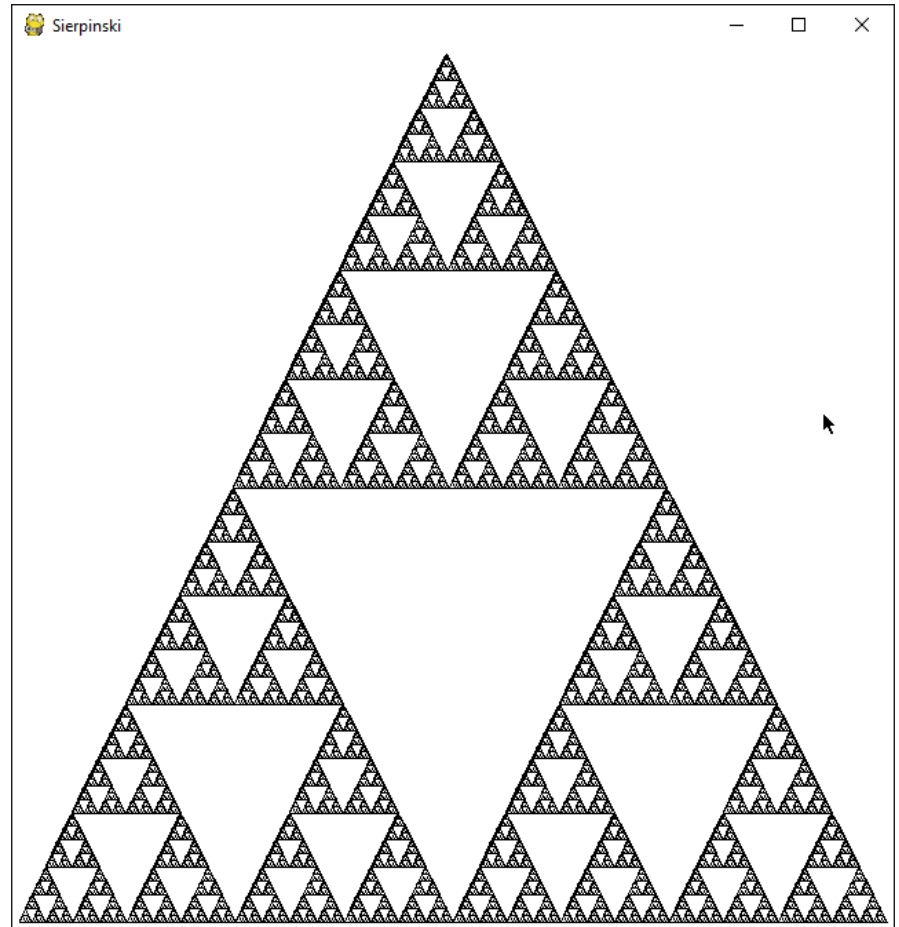
```
def koch2(a, order):  
    if order > 0:  
        for t in [60, -120, 60, 0]:  
            koch(a/3, order-1)  
            left(t)
```

```
koch(100, 0)  
pensize(3)  
#koch(100,1)  
koch2(100, 2)
```

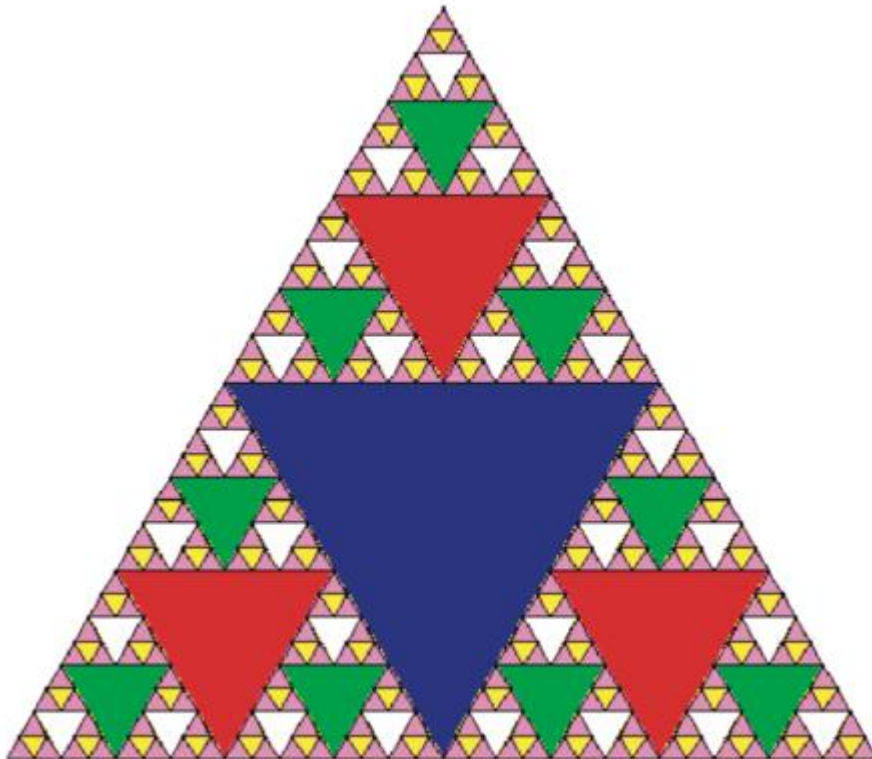


Sierpinski triangle

- ▶ The **Sierpiński triangle** is one of the most famous examples of a recursive geometric structure.
- ▶ It is formed by repeatedly subdividing an equilateral triangle into smaller ones and removing the central triangle at each iteration.



Sierpinski triangle



```
import turtle
```

```
def drawTriangle(points,color,myTurtle):  
    myTurtle.fillcolor(color)  
    myTurtle.up()  
    myTurtle.goto(points[0][0],points[0][1])  
    myTurtle.down()  
    myTurtle.begin_fill()  
    myTurtle.goto(points[1][0],points[1][1])  
    myTurtle.goto(points[2][0],points[2][1])  
    myTurtle.goto(points[0][0],points[0][1])  
    myTurtle.end_fill()
```

```
def getMid(p1,p2):  
    return ( (p1[0]+p2[0]) / 2, (p1[1] + p2[1]) / 2)
```

```
def sierpinski(points,degree,myTurtle):  
    colormap = ['blue','red','green','white','yellow',  
               'violet','orange']  
    drawTriangle(points,colormap[degree],myTurtle)  
    if degree > 0:  
        sierpinski([points[0],  
                    getMid(points[0], points[1]),  
                    getMid(points[0], points[2])],  
                    degree-1, myTurtle)  
        sierpinski([points[1],  
                    getMid(points[0], points[1]),  
                    getMid(points[1], points[2])],  
                    degree-1, myTurtle)  
        sierpinski([points[2],  
                    getMid(points[2], points[1]),  
                    getMid(points[0], points[2])],  
                    degree-1, myTurtle)
```

```
myTurtle = turtle.Turtle()  
myWin = turtle.Screen()  
myPoints = [[-100,-50],[0,100],[100,-50]]  
sierpinski(myPoints,3,myTurtle)  
myWin.exitonclick()
```

Fractals in nature



Read lists – Until EOF

```
import sys
```

```
for line in sys.stdin:  
    numbers = line.split(" ")  
    lists = []  
    for number in numbers:  
        lists.append(int(number))  
    print(lists)
```

Read lists – Until EOF

```
import sys

for line in sys.stdin:
    lists=[int(number) for number in line.split(" ")]
    print(lists)
```

Read lists – N test cases

```
n = int(input())
for i in range(n):
    numbers = input().split(" ")
    lists=[]
    for number in numbers:
        lists.append(int(number))
    print(lists)
```

Read lists – N test cases

```
n = int(input())
for i in range(n):
    line = input()
    lists=[int(number) for number in line.split(" ")]
    print(lists)
```

Read lists – Until an empty line

```
import sys
for line in sys.stdin:
    if line.strip() == "":
        break
    numbers = line.split(" ")
    lists=[]
    for number in numbers:
        lists.append(int(number))
    print(lists)
```


Read lists – Until an empty line

```
import sys

for line in sys.stdin:
    if line.strip() == "":
        break
    numbers = line.split(" ")
    lists = [int(number) for number in numbers]
    print(lists)
```

Homework 1

<https://viskillz.inf.unideb.hu/prog/#/2022?week=P1041>

- ▶ Write a function named *count_of_squares()* that returns how many elements in a list are **perfect squares**.
- ▶ In the *main()* function:
 - ▶ Read list elements until the given condition (EOF, n test cases, or empty line)
 - ▶ Call *count_of_squares()* and print the result for each list.

Homework1

Sample Input

- ▶ 63 64 14 6 10 6 39 37 71
- ▶ 71 9 81
- ▶ 82 65 71 79 46 88

Output for Sample Input

- ▶ 1
- ▶ 2
- ▶ 0

Homework 2

<https://viskillz.inf.unideb.hu/prog/#/2022?week=P1051>

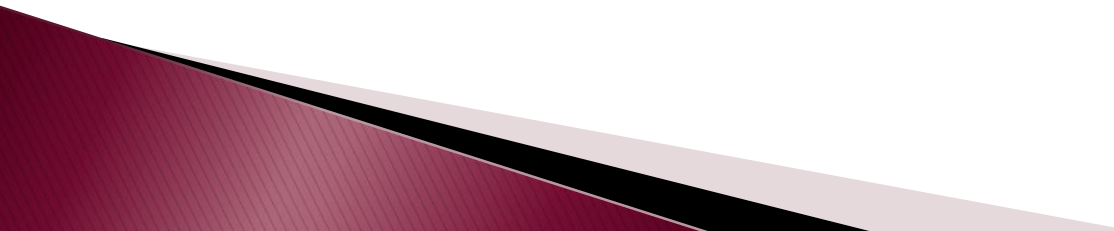
- ▶ Write a function named *delete_even_digits()* that removes all **even digit characters** from a given string and returns the result.
- ▶ In the *main()* function:
- ▶ Read strings until EOF / n test cases / empty line
- ▶ Call *delete_even_digits()* and print the modified string.

Homework2

Sample Input

- ▶ 0123456789
- ▶ a2bc4d1ef3gh
- ▶ aB1cD2eF3gH4

Output for Sample Input

- ▶ 013579
 - ▶ abcd1ef3gh
 - ▶ aB1cDeF3gH
- 
- A decorative gradient bar at the bottom left of the slide, transitioning from dark red to light grey.

Homework3

<https://viskillz.inf.unideb.hu/prog/#/?week=P1032>

- ▶ Write a function called *Pythagorean* that calculates the **hypotenuse** of a right triangle from its two sides (*a* and *b*).
- ▶ In the *main()* function:
- ▶ Read pairs of numbers until EOF / n test cases / empty line. (a b → in one line, separated by the space character)
- ▶ Call *Pythagorean()* and print the hypotenuse rounded to two decimal places.

Homework3

Sample Input

- ▶ 86.3 50.47
- ▶ 59.73 39.11
- ▶ 57.4 55.74
- ▶ 61.91 52.71
- ▶ 11.15 87.08

Output for Sample Input

- ▶ 99.97
- ▶ 71.40
- ▶ 80.01
- ▶ 81.31
- ▶ 87.79